The Autonomy Stack: Layers, Rates and Sensor Choice

Contents

1. Overview . . . 2
2. Sense, plan, act: one loop, as slow as its slowest step . . . 4
   - 2.1 The loop . . . 4
   - 2.2 How often the loop must look . . . 5
   - 2.3 Why the whole loop cannot run every 0.1 s . . . 7
3. Reactive control: sensors wired to the wheels . . . 7
   - 3.1 One sensor, one wheel . . . 7
   - 3.2 Two sensors, two wheels: steering to a light . . . 7
   - 3.3 What reacting alone cannot do . . . 10
4. Subsumption: reflexes in levels . . . 11
   - 4.1 Two levels for the room . . . 11
   - 4.2 The levels in the room with the person . . . 13
5. The layered stack: slow planning above fast reaction . . . 15
   - 5.1 Deliberative layers above reactive layers . . . 15
   - 5.2 The room's stack at 2.4 s . . . 15
   - 5.3 Why each layer runs at its rate . . . 17
   - 5.4 The layers in Nav2, Autoware and Apollo . . . 18
6. Choosing and placing sensors . . . 20
   - 6.1 Range: far enough to stop . . . 20
   - 6.2 Coverage: everywhere the robot can move . . . 21
   - 6.3 Redundancy: a second kind of sensor for the first one's blind spots . . . 22
   - 6.4 Compute budget: data the computer can process in time . . . 23
7. Summary . . . 25
8. Sources . . . 26
9. Key terms . . . 27

Where this fits:

- **Builds on:** Differential-drive kinematics; Global plan and local planner.
- **Leads to:** ROS 2 nodes, topics and messages; Nav2, start to finish.

1. Overview

Key point: A robot's software is split into layers that each run at their own rate: the mission and the route rarely, the local plan about 10 times a second, the wheel commands 20 times a second, all at once. One loop that waits for its slowest part sees the person in the room 0.6 s late and touches them; layered, the same robot reaches the door at 8.0 s.

The room robot already has every part it needs to reach the door:

- A* (G-2577) plans the route on the room's 0.5 m grid, 3.414 m;
- a particle filter (G-2492) run on the map estimates where it is;
- the local planner (G-2911) bends a timed plan round obstacles the map lacks, every 0.1 s;
- the trackers of the last chapter, such as Kanayama's tracker, turn a plan into wheel speeds and correct the robot as it drives.

Each part was built on its own. Nothing yet says which part runs when, how often, or which sensors feed them. The running scene shows why that matters. The robot starts at rest at (2.0, 1.0) facing 30° and must stop at the door pose (3.6, 3.5) facing 90°. At 2.4 s a person steps onto its route and stays, 0.465 m in front of the robot's rim, while the robot drives at 0.595 m/s.

Figure 1 runs the same robot twice. On the left the parts run one after another in one loop, and the loop waits for its slowest part, the whole-map step, which real stacks run about once a second. On the right the parts run as layers, each at its own rate. Watch the left robot keep driving its old plan after the person steps in.



Figure 1: The room seen from above, one frame per 0.2 s. Left: one loop that looks at the world once a second (at 0, 1, 2, 3 s) and drives the plan it made from the map in between (grey dotted). The person (pink) steps in at 2.4 s; the loop first sees them at 3.0 s, 0.113 m away at 0.591 m/s, brakes at 0.5 m/s^2, and touches them at 3.22 s at 0.48 m/s (red). Right: the layered stack of the earlier room drive, whose local planner looks every 0.1 s: its plan (orange) bends left round the person at 2.4 s, and the robot reaches the door at 8.0 s

The left robot detects the person 0.6 s after they step in, at 3.0 s, when the gap is only 0.113 m; braking needs 0.350 m, so it touches them at 3.22 s (notebook). The right robot reaches the door at 8.0 s, 0.14 m clear of the person. The opening questions of this Note:

1. **Why does one loop fail, and how often must the robot look?**
2. **How should the parts be arranged so that the slow ones do not hold up the fast ones?**
3. **Which sensors see the person in time, and where should they go?**

We answer them in this order:

- one loop that senses, plans and acts, and the rate it would need (Section 2);
- the opposite extreme, sensors wired straight to the wheels (Section 3);
- several such reflexes stacked in levels (Section 4);
- the layered stack that combines a plan with fast reactions, its rates, and the same layers in Nav2, Autoware and Apollo (Section 5);
- choosing and placing the sensors that feed it (Section 6).

2. Sense, plan, act: one loop, as slow as its slowest step

> **Key point:** Sense–plan–act runs sensing, modelling, planning and acting in order, in one loop, so the robot looks again only when the whole loop is done. To stop for the person it must look at least every 0.187 s, but a whole-map step runs about once a second.

2.1 The loop

The first opening question asks why one loop fails. The simplest way to join the parts is to run them one after another, in a repeating loop:

1. **sense:** read every sensor, the laser scan and the wheels;
2. **plan:** update the map and the robot's pose, then plan a route over the whole map;
3. **act:** drive that plan until the next pass.

Figure 2 steps through the opening's left-hand robot, whose loop looks once a second, from 1.9 to 3.22 s. Watch which step is lit when the person steps in at 2.4 s.



Figure 2: Animation: the opening's one loop, step by step. Top: sense (read every sensor), plan (the map, the pose, the whole route), act (drive the plan), then back to sense one second later; the step running is lit. Bottom: a time line with the loop's looks (green, 2.0 and 3.0 s), the person stepping in (pink, 2.4 s) and the touch (red, 3.22 s). At 2.0 s the loop looks and plans with nobody on the route, then acts on that plan; at 2.4 s the person steps in 0.465 m ahead while the loop is acting, not looking; at 3.0 s it sees them 0.113 m ahead and plans to brake; at 3.22 s it touches them

Running sensing, planning and acting in this fixed order, with the whole plan made from a model of the world before the robot moves, is **sense–plan–act** (G-3211), also called

deliberative control. Sense–plan–act was the first way robots were built: Shakey, the 1960s robot whose planner was STRIPS, ran this way (Chernova, CS 7633 slides).

An everyday picture: a driver who glances at the road, then closes their eyes to plan the whole trip again before looking up. Between two glances, whatever happens on the road goes unseen. The loop's period, the time from one look to the next, is set by its slowest step.

2.2 How often the loop must look

The person steps in 0.465 m ahead of the robot's rim. In the worst case they step in just after a look, so the robot drives on without seeing them for one whole period and only then starts to brake. With:

- v, the robot's speed when the person steps in, 0.595 m/s;
- a, the hardest the robot can brake, 0.5 m/s^2;
- g, the gap from the robot's rim to the person, 0.465 m;
- T, the loop's period, the time between two looks, in s;
- d, the distance the robot covers from the person stepping in to standing still, in m.

The distance has two parts: the robot drives on at the speed v for up to one period before it sees the person, then it brakes over the stopping distance derived for this robot, the speed squared over twice the braking:

$$d = v\,T + \frac{v^2}{2a}$$

The braking part with our numbers:

$$\frac{0.595^2}{2 \times 0.5} = 0.354 \text{ m}$$

The robot stops in time only if d is less than the gap g. Our numbers first, then the same steps in symbols:

$$0.595\,T + 0.354 < 0.465$$

$$v\,T + v^2/(2a) < g$$

$$0.595\,T < 0.465 - 0.354$$

$$v\,T < g - v^2/(2a)$$

$$T < (0.465 - 0.354)/0.595$$

$$T < \frac{g - v^2/(2a)}{v}$$

$$T < 0.111/0.595 = 0.187 \text{ s}$$

The loop must therefore look at least every 0.187 s, about 5.3 times a second. A check at an easy value: with $T = 0$, a robot that sees the person the instant they step in needs only the 0.354 m of braking, less than 0.465 m, so it stops. Figure 3 plots d for every period.

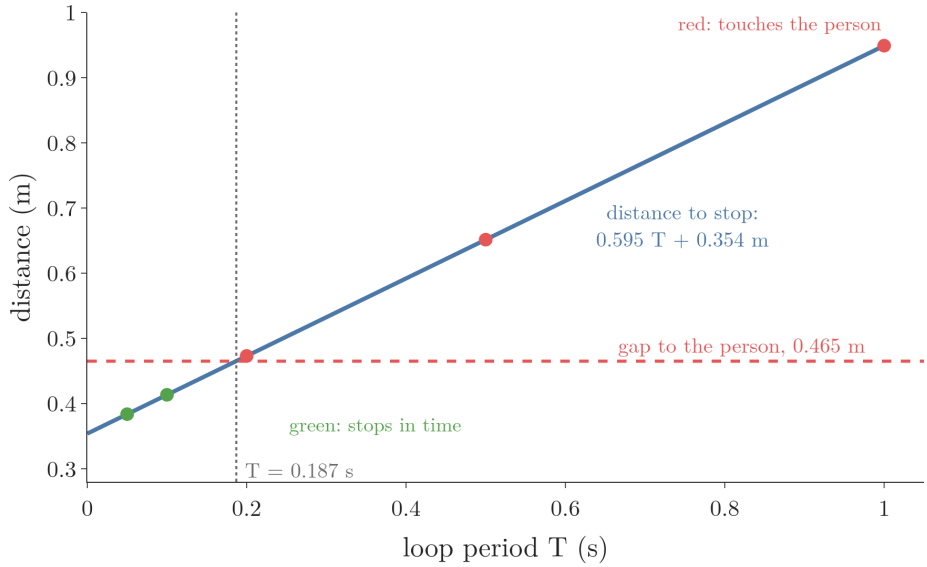


Figure 3: The distance the robot covers before it stands still, 0.595 T + 0.354 m (blue), against the loop period T, with the 0.465 m gap to the person (red, dashed). The line crosses the gap at T = 0.187 s. Dots: periods of 0.05 and 0.1 s stop in time (green); 0.2, 0.5 and 1 s touch the person (red)

Loop period T	Driven before the look	Plus braking	Result
1 s	0.595 m	0.949 m	touches
0.5 s	0.297 m	0.651 m	touches
0.2 s	0.119 m	0.473 m	touches
0.1 s	0.059 m	0.413 m	stops
0.05 s	0.030 m	0.384 m	stops

The run of Figure 1 is less severe than the worst case: its 1 s loop looks at 3.0 s, 0.6 s after the person steps in, and it still touches them at 3.22 s. The same loop looking every 0.1 s brakes at 2.4 s and stands still at 3.59 s, 0.112 m from the person (notebook). A loop that looks every 0.1 s would therefore be fast enough; the question is whether the whole loop can run that often.

2.3 Why the whole loop cannot run every 0.1 s

The obvious remedy is to run the whole loop every 0.1 s. The plan step makes that impossible. Updating a map of the whole area and planning a route across it is the slowest work a navigation robot does, which is why the global plan is refreshed rarely, every 2 s in a ROS benchmark. Nav2, the ROS 2 navigation stack, updates its whole-map costmap, a grid of the area that marks where obstacles are, once a second and asks for a new global plan once a second (nav2_params.yaml; Nav2 default tree).

A loop that contains this step therefore looks about once a second, more than five times too slowly for the 0.187 s that Section 2.2 requires. Sense–plan–act works only when the world can be modelled accurately and hardly changes while the robot carries out its plan (Chernova, CS 7633 slides). The person broke the second condition in less than one pass. The robot needs a part that reacts within one sensor reading, without waiting for a plan.

3. Reactive control: sensors wired to the wheels

> **Key point:** Reactive control maps sensor readings straight to wheel speeds, with no map and no plan, so the robot reacts within one reading. Braitenberg's vehicles show what two sensors and two wires can do: wired one way the robot steers to a light, but it cannot sense the table in its path.

3.1 One sensor, one wheel

The opening's first question needs a robot that reacts within one reading, not one plan. The opposite of a long chain of steps is no chain at all: wire a sensor straight to a motor. A designer who chains many steps must foresee every situation the robot can meet and write a response for each; a simple wire foresees nothing, yet the robot still behaves sensibly.

The simplest case is one light sensor wired to one wheel: the more light, the faster the wheel. In the shade the vehicle moves slowly, in bright light it moves fast, and so it spends most of its time in the dark. An onlooker would say it fears the light, but nothing inside it knows about fear; the behaviour comes from the wire alone. With the wire reversed, so that more light means slower, the same vehicle lingers in the light instead.

Control that maps the present sensor readings straight to motor commands, with no model of the world and no plan, is **reactive control** (G-3212). One such mapping from sensor readings to motor actions, made for one task, is a **behaviour** (G-3213) (Chernova, CS 7633 slides). A reflex is the everyday example: a hand pulls back from a hot stove before any thought about stoves. Because nothing waits for a plan, a reactive robot reacts within one sensor reading. One sensor, though, can only change the speed, not the direction.

3.2 Two sensors, two wheels: steering to a light

One sensor can only change the speed. With two light sensors at the front and the two wheels of our robot, the wiring can also steer. Figure 4 shows the four ways to wire them:

- the wires go to the wheel on the same side, or cross over;

- each wire speeds its wheel up with more light (+) or slows it down (−).

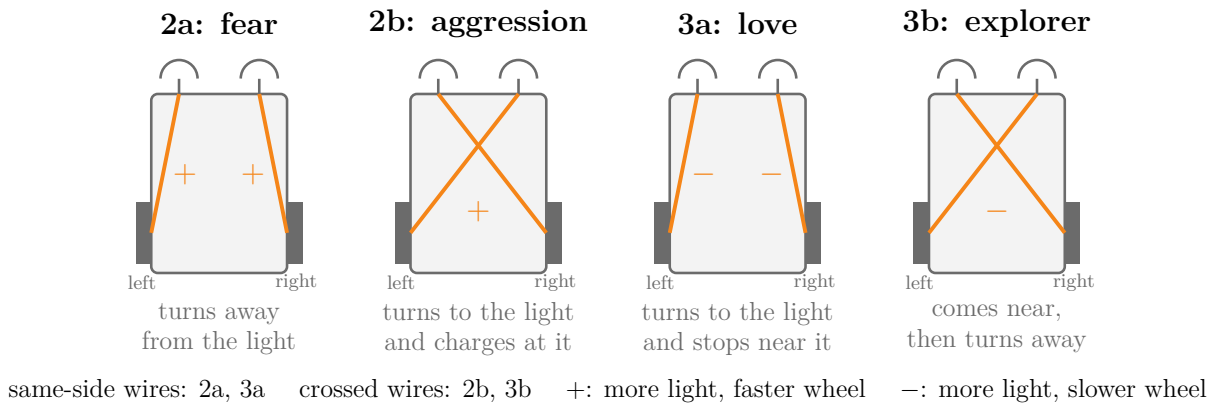


Figure 4: Braitenberg's vehicles 2 and 3, seen from above: two light sensors at the front, two wheels at the back, and the wires between them. 2a fear: same-side wires, more light makes a wheel faster; it turns away from the light. 2b aggression: crossed wires, faster; it turns to the light and charges at it. 3a love: same-side wires, more light makes a wheel slower; it turns to the light and stops near it. 3b explorer: crossed wires, slower; it comes near, then turns away

A vehicle of one or two light sensors wired straight to its wheels is a **Braitenberg vehicle** (G-3214), after the thought experiments of Valentino Braitenberg's book *Vehicles*; the four wirings are his vehicles 2a, 2b, 3a and 3b (Wikipedia, "Braitenberg vehicle").

Why the wiring steers, on our robot. Our robot starts at (2.0, 1.0) facing 30°. We put a lamp at the door pose, (3.6, 3.5), ahead and to the left, so that a vehicle that seeks light has the room's goal as its target. We give the robot two light sensors on its rim, 45° either side of its heading. Our model of a reading, a number from 0 to 1, is the lamp's brightness at the sensor times how squarely the sensor faces the lamp. With:

- d_s, the distance from the sensor to the lamp, in m;
- D_0, the distance at which the brightness falls to half, 1 m (our choice);
- α, the angle between the direction the sensor faces and the direction to the lamp, in degrees;
- s, the sensor's reading, a number from 0 to 1.

$$s = \frac{1}{1 + (d_s/D_0)^2} \times \frac{1 + \cos\alpha}{2}$$

The first part is 1 at the lamp and falls to one half at 1 m; the second is 1 when the sensor faces the lamp, one half when the lamp is square to its side and 0 when the lamp is behind it.

The left sensor faces 75°, 2.731 m from the lamp and 19.2° off it; the right sensor faces −15°, 2.902 m away and 77.09° off it (notebook):

$$s_L = \frac{1}{1 + 2.731^2} \times \frac{1 + \cos 19.2^\circ}{2}$$

$$= 0.1182 \times 0.9722 = 0.1149$$

$$s_R = \frac{1}{1 + 2.902^2} \times \frac{1 + \cos 77.09^\circ}{2}$$

$$= 0.1061 \times 0.6117 = 0.0649$$

The left sensor reads more light, as expected, because the lamp is to the left. We now wire vehicle 3a: same side, more light makes the wheel slower. With:

- s_L and s_R, the left and right readings, 0.1149 and 0.0649;
- k, the wire's gain, how strongly a reading slows its wheel, 2 (our choice);
- $v_{\max}$, a wheel's top speed, 0.6 m/s;
- v_L and v_R, the left and right wheel speeds, in m/s.

$$v_L = v_{\max}\,(1 - k\,s_L)$$

$$v_R = v_{\max}\,(1 - k\,s_R)$$

With our numbers:

$$v_L = 0.6 \times (1 - 2 \times 0.1149)$$

$$= 0.6 \times 0.7702 = 0.4621 \text{ m/s}$$

$$v_R = 0.6 \times (1 - 2 \times 0.0649)$$

$$= 0.6 \times 0.8702 = 0.5221 \text{ m/s}$$

The right wheel runs faster, so the robot turns left, towards the lamp. How fast it turns is the wheels' difference over the 0.2 m wheel separation, the angular velocity (G-2292) formula:

$$\omega = (0.5221 - 0.4621)/0.2$$

$$= 0.300 \text{ rad/s}$$

A check at an easy value: with the lamp straight ahead, both sensors read the same, both wheels turn at the same speed, and the robot drives straight at the lamp. As it approaches, both readings grow and both wheels slow, so it stops near the lamp.

Figure 5 runs all four wirings on an open floor with one lamp, so that only the wiring decides each path. Each vehicle starts 1.62 m from the lamp with the lamp ahead and to the left, and its wheels follow the wires at once, as in Braitenberg's thought experiments. Watch which way each one turns in its first second.

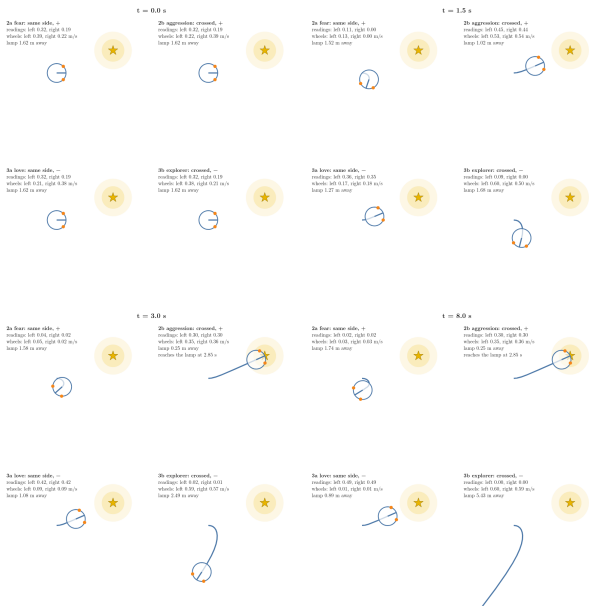


Figure 5: Animation: the four vehicles of Figure 4 on an open floor, each starting 1.62 m from the lamp (yellow star) with the lamp ahead and to the left; orange dots are the two light sensors, and each panel prints the readings, the wheel speeds and the distance to the lamp; one frame per 0.1 s for 8 s. 2a fear turns away and slows almost to a stop, 1.74 m away. 2b aggression turns to the lamp, speeds up and reaches it at 2.85 s. 3a love turns to it and slows, nearly still 0.89 m away at 8 s. 3b explorer comes within 1.48 m, then turns away and leaves at full speed

In the last frame, 3a has nearly stopped 0.89 m from the lamp and comes to rest 0.87 m from it; 2b has driven into the lamp at 2.85 s; 2a and 3b have turned away (notebook). Two wires and a sign decide whether the same robot seeks the light or flees it. Vehicle 3a seeks the lamp and slows near it; the next test is the room.

3.3 What reacting alone cannot do

Vehicle 3a turns to the lamp within one reading, the response time the opening requires. Figure 6 puts it in the room with the lamp at the door pose and no person, with our robot's real limits: each wheel at most 0.6 m/s and the speed changing by at most 0.5 m/s^2. Watch where its straight run at the lamp passes the table's corner.

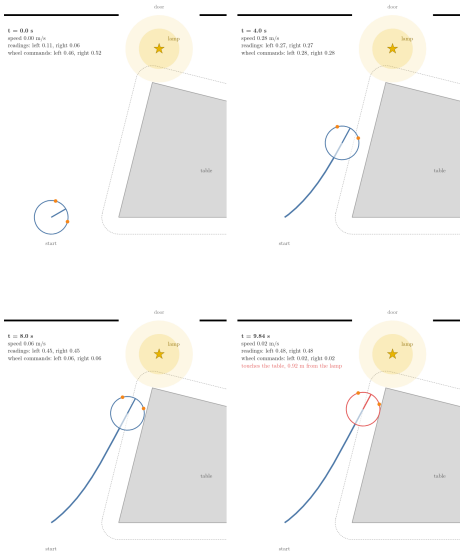


Figure 6: Animation: vehicle 3a in the room, starting at rest at (2.0, 1.0) facing 30°, the lamp (yellow star) at the door pose; one frame per 0.2 s. The vehicle turns to the lamp within its first second; from 4 s both readings are equal (0.27) and it drives straight at the lamp, slowing as the light grows. Its straight line passes the table's corner, and at 9.84 s its rim touches the table's edge (red), 0.92 m from the lamp

The vehicle touches the table at 9.84 s, moving at 0.02 m/s, 0.92 m from the lamp (notebook). No sensor in its wiring detects the table, so the table has no effect on its motion. A purely reactive robot cannot plan a route, build a map, check how well it is doing or choose which behaviour to use (Chernova, CS 7633 slides). The first gap can be closed with more sensors and more behaviours, one for the light and one for obstacles; the next section stacks them.

4. Subsumption: reflexes in levels

> **Key point:** Subsumption runs several behaviours at once in levels, the lowest built first; a higher level replaces a lower level's command when it has one, but a safety reflex such as "halt" is never overruled. In the room it never touches anything, yet it stops 0.283 m from the person and stays there, 1.748 m from the lamp.

4.1 Two levels for the room

The vehicle of Figure 6 reacts fast but senses only the lamp. The opening's question becomes: how can several fast behaviours, each a mapping from readings to wheel commands, share one pair of wheels? Brooks's answer was to build the robot in levels (Dean, CS148 notes; Wikipedia, "Subsumption architecture"):

1. Build the lowest level first, a robot that only avoids things, and test it on its own.

2. Add the next level on top, a robot that goes somewhere, and let it use the lower level's work.
3. Where both send wheel commands, let the higher level's command replace the lower one's, except for the reflexes that keep the robot safe.

For the room we build two levels (Figure 7). Level 0, avoid, has a ring of 36 range readings, one every 10°, taken every 0.1 s:

- **collide** halts the robot, forward speed 0, when a reading within 30° of straight ahead is under 0.42 m;
- **push** (Brooks's feelforce) turns every reading under 0.5 m into a push away from it, harder when nearer;
- **runaway** turns the robot along the push and drives away.

Level 1, go to the light, is vehicle 3a of Section 3.2 with one addition: it also turns away from the push, faster the harder the push, with a turn gain of 2. Its command replaces runaway's.

The ring of 36 readings, collide's 30° cone, the push's 0.5 m reach and the turn gain of 2 are our own settings for the room; Section 4.2 varies the last two. Only collide's 0.42 m follows from a rule, shown next.

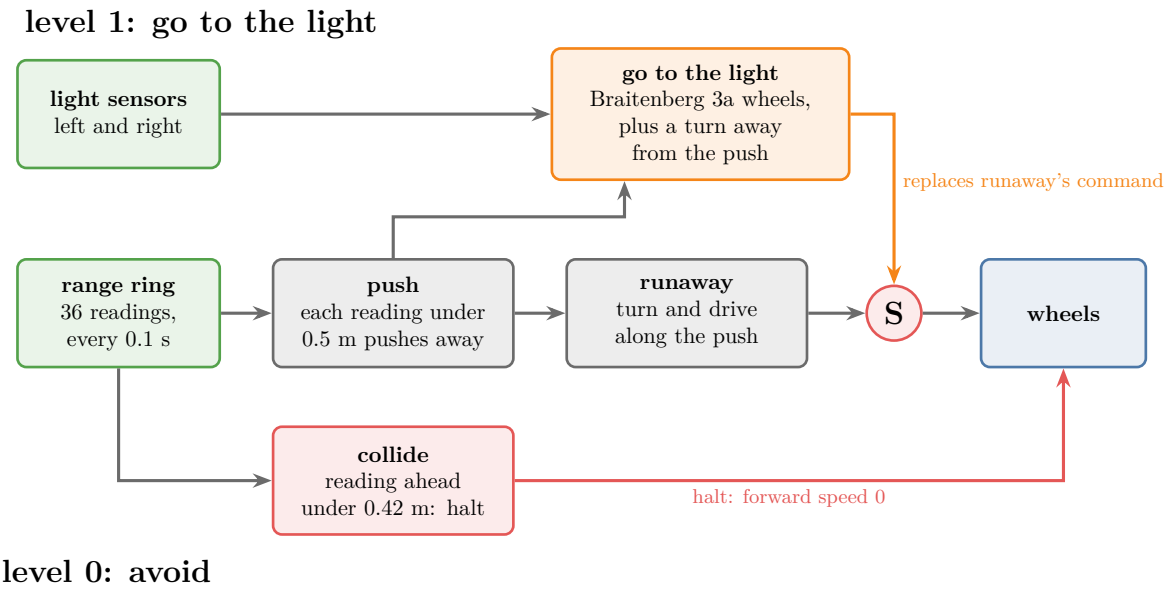


Figure 7: The two levels. Level 1 (top): the light sensors feed "go to the light", the Braitenberg 3a wheels plus a turn away from the push; its command replaces runaway's at the suppression node S. Level 0 (middle and bottom): the range ring feeds "push" (each reading under 0.5 m pushes away) and "collide" (a reading ahead under 0.42 m: halt); push feeds runaway (turn and drive along the push), whose command reaches the wheels through S; collide's halt goes straight to the wheels and sets the forward speed to 0

The 0.42 m halt distance comes from the formula of Section 2.2 at the robot's top speed with the 0.1 s look. One look's driving plus the braking:

$$0.6 \times 0.1 + \frac{0.6^2}{2 \times 0.5}$$

$$= 0.06 + 0.36 = 0.42 \text{ m}$$

The standard terms: a design in which behaviours run in parallel in levels, each level adding a competence on top of the ones below and taking over their commands when it has a command to send, is the **subsumption architecture** (G-3215), from Brooks (1986). A higher level takes over in two ways, **suppression and inhibition** (G-3216): suppression replaces the input of a lower module with the higher level's own signal, as level 1 does to runaway's command at S; inhibition blocks a lower module's output from going anywhere (Dean, CS148 notes). A course version writes the whole arbitration as one priority chain: if the top level's condition holds, use its command; else if the next one's holds, use that; down to level 0 (Jenkins, CSCI 1480 slides). The next question is what the two levels do when the person steps in.

4.2 The levels in the room with the person

The opening's person steps in at 2.4 s: do the two levels keep the robot clear of them and still get it to the door? Figure 8 drives the two levels from the start. The right panel lights the module whose command reaches the wheels. Watch the readings that turn red when the person comes within 0.5 m, and what the robot does after collide fires.

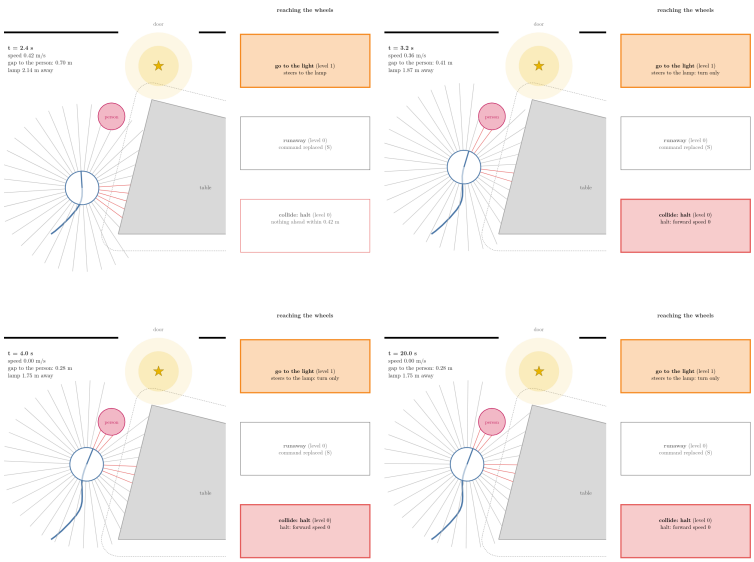


Figure 8: Animation: the two levels of Figure 7 in the room; one frame per 0.2 s to 6 s, then 20 s. Left: the robot with its 36 range readings (red where a reading is under 0.5 m and pushes), the lamp (yellow star) and the person (pink, from 2.4 s). Right: the modules, lit when their command reaches the wheels. Level 1 drives until 3.2 s, when the person is 0.41 m ahead and collide halts the robot; level 1 still turns it, but the lamp turns it towards the person and the push turns it away. From 3.91 s it stands still, 0.28 m from the person and 1.75 m from the lamp, and at 20 s nothing has changed

The levels did what they were built for: collide fires at 3.2 s with the person 0.411 m ahead, the robot stands still from 3.91 s, and it never touches anything (notebook). But it stays there. At 20 s it is still 0.283 m from the person and 1.748 m from the lamp. The lamp's pull turns it towards the person, the push turns it away by the same amount, and collide holds the forward speed at zero: the same balance of pull and push that traps a robot in a potential field's local minimum.

The stop is not a matter of tuning. With the push's reach set to 0.4, 0.5 or 0.6 m and its turn gain to 1, 2 or 3, nine runs in all, none came closer than 1.146 m to the lamp in 20 s, and none came closer than 0.139 m to anything (notebook). Wikipedia lists the architecture's two weaknesses: choosing between behaviours through many suppression and inhibition links is hard to design, and without memory or a symbolic model it cannot build maps (Wikipedia, "Subsumption architecture"). Without a map, the robot has no way to know that the way round the person is to the left. The reflexes keep the robot safe; choosing the route requires the plan that Section 2 could not run fast enough.

5. The layered stack: slow planning above fast reaction

> **Key point:** The layered stack runs the slow, long-range layers (the mission and the route) rarely and the fast, short-range layers (the local plan and the wheel commands) often, all at the same time, so a new reading reaches the wheels in one fast cycle while the route still comes from the map. In the room the person's scan reaches the wheels within the 0.1 s cycle at 2.4 s, and the robot reaches the door at 8.0 s.

5.1 Deliberative layers above reactive layers

Sections 2 to 4 leave the opening's second question half answered. Sense–plan–act has a plan but reacts once a second; reactive control reacts within a reading but has no plan. The answer is to run both at once, each at its own speed: slow planning layers on top, fast reacting layers below, the lower layers working from the upper layers' latest output instead of waiting for them. Such a hybrid deliberative and reactive design is the best general architecture for robots, because the planning work runs separately from the reactions while the reactions keep their speed (Chernova, CS 7633 slides).

A robot design that combines a slow planning part, which reasons over a model of the world, with a fast reactive part that acts on the latest readings, the two running at the same time, is a **hybrid deliberative/reactive architecture** (G-3217). Navigation software splits it into four layers, the levels of planning met with the local planner, each feeding the one below:

1. **mission:** where to go, such as the door pose, and the route there, the job of route planning (G-2870), also called mission planning;
2. **behaviour:** which task to do now: follow the route, stop, give way (here the word means a task, not the sensor-to-wheel behaviour of Section 3.1), the behavioural layer (G-2912);
3. **motion:** the path and the timed plan, from the global planner (G-2910) over the whole map and the local planner (G-2911) over the next few metres;
4. **control:** the wheel commands that follow the plan, from the controller (G-2947).

The whole of a robot's software arranged this way, sensing at the bottom and these layers above it, each running at its own rate, is the robot's **autonomy stack** (G-3210). A car trip has the same shape: the destination is chosen once, the route is checked now and then, and the steering is corrected every moment. The same shape appears inside control, where a cascade runs a fast inner loop under a slower outer one. The next section gives the room's own stack, with its numbers.

5.2 The room's stack at 2.4 s

The opening's right-hand robot reached the door at 8.0 s with a layered stack. Its layers, with what each takes in, what it hands down and how often it runs; the rates are our choice for the room robot:

Layer	Takes in	Hands down	Runs
mission	the goal	the door pose (3.6, 3.5), 90°	once
behaviour	the robot's progress	the task: follow the route	every 0.1 s
global planner	the map	the A* route, 3.414 m	once
local planner	the route, the scan, the pose	a timed band over 1.5 m	every 0.1 s
controller	the band, the wheel readings	wheel speeds	every 0.05 s
sensors	the room	a scan; wheel readings	every 0.1 s; every 0.05 s

The local planner's 0.1 s and the band are the local planner's own; the controller's 0.05 s is the 20 Hz that the timed tracker took from Nav2's controller; a scan every 0.1 s is a laser scanner's own rate (Section 6.1). Figure 9 follows one reading up the stack and one command down. Each layer's dot lights in the frames where it runs: the controller every frame, the local planner and the behaviour layer every other frame, the mission and the global planner only at 0 s. Watch the frames at 2.40 s.

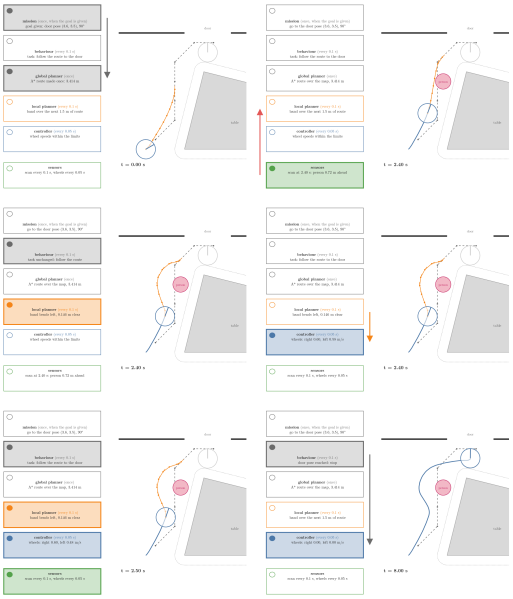


Figure 9: Animation: left, the room robot's layers with their rates; a layer's dot lights in the frames where it runs. Right: the room. At 0 s the mission gives the door pose and the global planner makes the A* route, once. From 2.30 s, one frame per 0.05 s. At 2.40 s three steps: the scan that first shows the person (0.72 m ahead of the robot's centre) goes up to the local planner (red arrow); the local planner bends its band left, 0.146 m clear of the person; the new band goes down to the controller (orange arrow), which sends wheel speeds right 0.60, left 0.58 m/s. The controller then sends left 0.53, 0.48 and 0.43 m/s at 2.45, 2.50 and 2.55 s with the right wheel at its 0.60 m/s limit. Last frame, 8.0 s: the behaviour layer sees the door pose reached and stops the robot

At 2.40 s the scan shows the person 0.72 m ahead of the robot's centre. In the same 0.1 s cycle the local planner bends its band from overlapping the person by 0.218 m to passing 0.146 m clear. The controller cannot meet the band's request at once: the band asks for 0.44 m/s and 2.07 rad/s, which would need the right wheel at 0.65 m/s, above its 0.6 m/s limit. The controller therefore keeps the right wheel at 0.60 m/s and slows the left from 0.58 to 0.43 m/s over the next 0.15 s; the robot turns left while it slows from 0.595 to 0.545 m/s by 2.5 s (notebook). Neither the mission nor the global plan runs again. The robot reaches the door at 8.0 s, 0.14 m clear of the person, as the local planner found. Why these rates, and not others, is the next point.

5.3 Why each layer runs at its rate

Figure 9's layers run at rates from once to 20 times a second. Each rate follows from how far the robot travels before the layer's answer is out of date. At the robot's top speed, 0.6 m/s:

Period	The robot travels	Fine for
1 s	0.6 m	a route over the room: walls and tables do not move
0.1 s	0.06 m	the local plan: Section 2.2 asked for 0.187 s or less
0.05 s	0.03 m	the wheel commands: the robot's own errors grow fast

- **The local planner** must see a newcomer within the 0.187 s of Section 2.2, so 0.1 s is enough; a slower layer would repeat the 1 s loop's failure.
- **The controller** corrects the robot's own drift from the plan, and a feedback loop works only when it ticks fast compared with how fast the robot responds; choosing the sample rate gives the rule. So it runs faster than the planner above it.
- **The mission and the global plan** change only when the goal or the map changes, so running them more often gains nothing and takes computing time the fast layers need.

The slow layers can therefore be slow without harm: a lower layer always works from the latest output of the layer above, and the response to the person never waited for the global plan.

5.4 The layers in Nav2, Autoware and Apollo

Three real stacks have the same four layers. Their rates, read from the code on 2026-10-12, are in Figure 10.

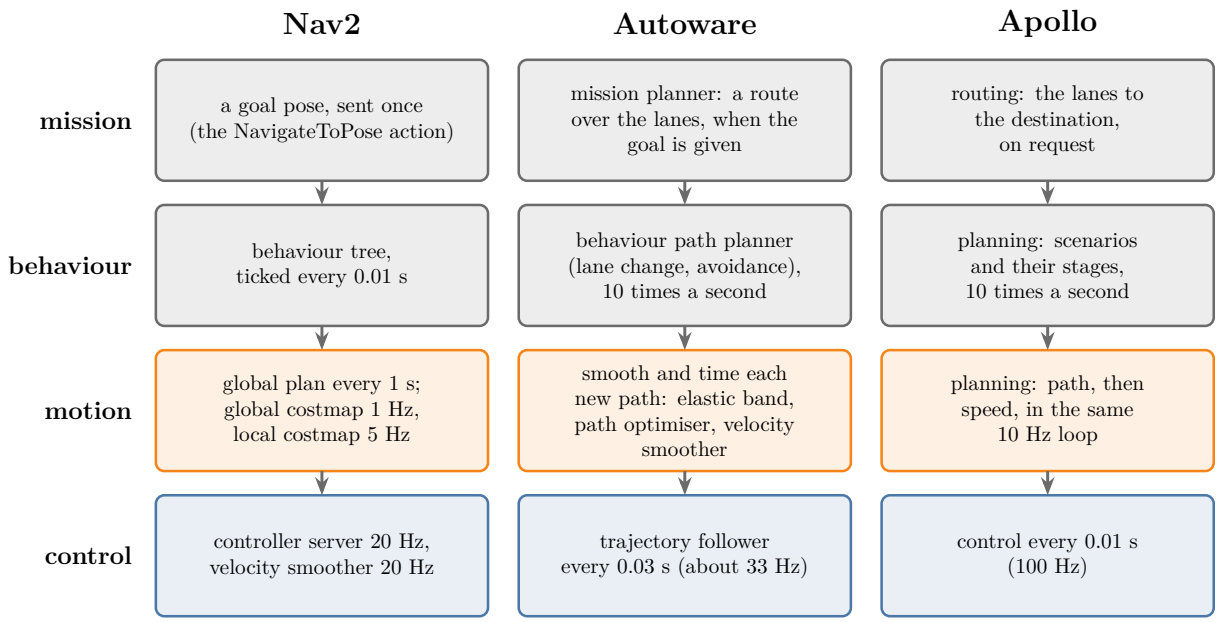


Figure 10: The four layers in three real stacks, rates read from the code. Nav2: a goal pose sent once (the NavigateToPose action); a behaviour tree ticked every 0.01 s; a global plan every 1 s, the global costmap updated at 1 Hz and the local costmap at 5 Hz; the controller server and the velocity smoother at 20 Hz. Autoware: the mission planner's route over the lanes, when the goal is given; the behaviour path planner (lane change, avoidance) 10 times a second; smoothing and timing each new path (elastic band, path optimiser, velocity smoother); the trajectory follower every 0.03 s. Apollo: routing on request; planning's scenarios and stages, 10 times a second; path then speed in the same 10 Hz loop; control every 0.01 s

- **Nav2** (`ros-navigation/navigation2`, `nav2_bringup/params/nav2_params.yaml` and the default tree `navigate_to_pose_w_replanning_and_recovery.xml`). The goal pose arrives once. Nav2 writes its behaviour layer as a behaviour tree, a tree of task steps that it checks repeatedly (behaviour trees and recoveries come in a later Note); the tree is checked every 10 ms (`bt_loop_duration: 10`), but its `RateController` asks the planner server for a new global plan only once a second (`hz="1.0"`), and the global costmap updates at 1 Hz. The local costmap updates at 5 Hz; the controller server and the velocity smoother run at 20 Hz (`controller_frequency: 20.0`, `smoothing_frequency: 20.0`).
- **Autoware** (`autowarefoundation/autoware_launch`). The mission planner computes a route over the lanes when the goal is given. The behaviour layer hands down a path with points about 1 m apart; the motion layer turns it into a trajectory with points about 0.1 s apart; a separate validation step checks the trajectory before control gets it (Autoware planning design). The behaviour path planner runs at 10 Hz (`planning_hz: 10.0`) and the trajectory follower every 0.03 s (`ctrl_period: 0.03`).
- **Apollo** (`ApolloAuto/apollo`). Routing finds the lanes to the destination on request.

The planning module picks a scenario and its stage, then plans path and speed, in a 10 Hz loop (`planning_loop_rate`, 10); control runs every 10 ms (`control.dag`, `interval: 10`).

In all three the top layer plans far ahead and changes rarely, and the bottom layer looks a short way ahead and runs often. The stack can only react to what reaches it, so the last question is what its sensors see.

6. Choosing and placing sensors

> **Key point:** A sensor must reach far enough to stop (range), look everywhere the robot can move (coverage), be backed by a sensor of another kind where it is blind (redundancy), and send no more data than the computer can process in time (compute budget). Our robot needs to see 0.67 m from its centre; a 240° laser scanner reaches 5.6 m but leaves 120° behind it blind.

6.1 Range: far enough to stop

The opening's third question is which sensors see the person in time. The first number is how far the sensor must reach. The stack looks every 0.1 s (Section 5.3), and at the top speed of 0.6 m/s the robot then needs, by the formula of Section 2.2:

$$0.6 \times 0.1 + \frac{0.6^2}{2 \times 0.5}$$

$$= 0.06 + 0.36 = 0.42 \text{ m}$$

The 0.42 m is measured from the rim; the sensor sits at the centre, 0.25 m further back:

$$0.42 + 0.25 = 0.67 \text{ m}$$

A sensor whose maximum range (G-2373), the longest distance it can report, is under 0.67 m sees obstacles too late to stop. Range has a cost. An ultrasonic sensor must wait for its echo, so a longer range means fewer readings a second (Correll §5.4): to hear an echo from 5 m, sound travels 10 m out and back, at about 300 m/s:

$$10/300 = 0.0333 \text{ s}$$

$$1/0.0333 = 30 \text{ readings per second}$$

A laser scanner such as the Hokuyo URG-04LX-UG01 measures 0.02 to 5.6 m with one scan every 0.1 s (Hokuyo specification), far more than the 0.67 m needed. Nav2 marks obstacles in its costmaps only up to 2.5 m and clears free space up to 3.0 m (`obstacle_max_range`,

`raytrace_max_range` in nav2_params.yaml), because farther readings are not needed for the next few metres of driving. Sufficient range is of no use in a direction the sensor does not cover.

6.2 Coverage: everywhere the robot can move

Range says how far; coverage says in which directions. Figure 11 puts two real sensors on the robot at 2.4 s. Look at where each one is blind.

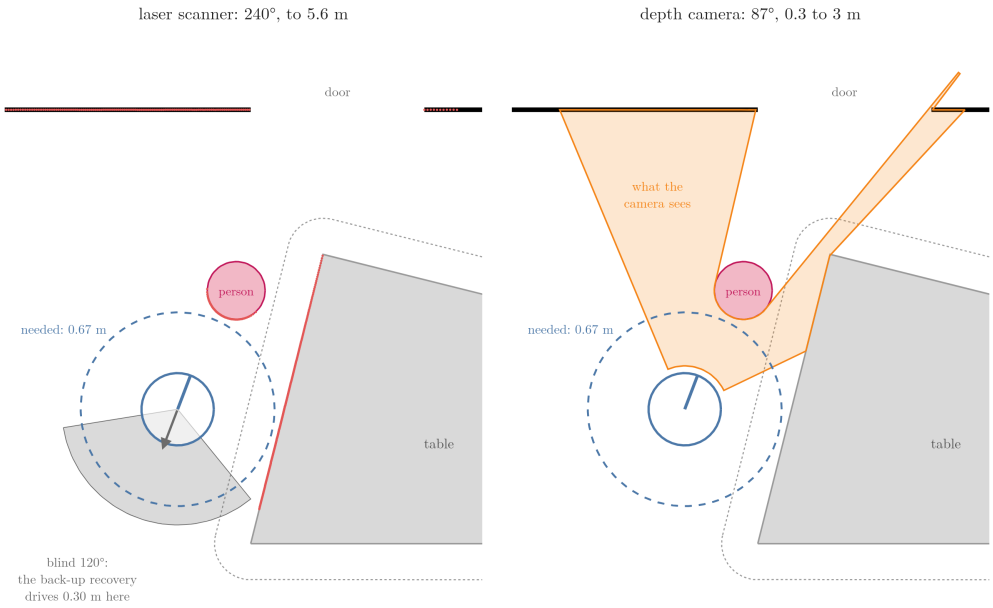


Figure 11: Two real sensors on the robot at 2.4 s, when the person steps in; the dashed circle is the 0.67 m the robot needs to see. Left: a laser scanner like the Hokuyo URG-04LX-UG01 at the robot's centre, facing forward: 240° in 0.352° steps (682 readings) to 5.6 m; red dots are where the readings stop, on the person, the table's edge and the top wall. The grey wedge is the 120° behind it that it never sees, where Nav2's default back-up recovery drives 0.30 m (arrow). Right: a depth camera like the RealSense D435: what it sees (orange) within its 87° view from 0.3 to 3 m, with the person's shadow behind them

The directions a sensor can measure, as an angle around the robot, is its **field of view** (G-3218); the field of view and the range together decide what the sensor covers. In Figure 11 both sensors see the person, 5.4° to the right of straight ahead. Each has a gap:

- **The scanner** covers 240° and is blind in the 120° behind it (Hokuyo specification). Nav2's default tree backs the robot up 0.30 m at 0.15 m/s as a recovery when it cannot make progress (Nav2 default tree), 2 s of driving into that blind wedge.
- **The camera** covers only 87° and sees nothing nearer than about 0.3 m (RealSense D435 specification), so an obstacle beside the robot, or very close in front, is invisible to it.

- **Height** matters too. A laser scanner measures in one plane, at its own height (sensor_msgs/LaserScan, "single scan from a planar laser range-finder"); a table top above that plane returns nothing. Nav2's local costmap is therefore a stack of 16 layers of 0.05 m cubes (`z_voxels: 16`, `z_resolution: 0.05`), so that a 3D sensor can mark obstacles above the scanner's plane.

Placement follows: a scanner on top of the robot, at its centre, sees all round if nothing on the robot blocks it; one at the front edge sees at most the half circle ahead. Coverage counts directions; it says nothing about surfaces a sensor cannot measure.

6.3 Redundancy: a second kind of sensor for the first one's blind spots

Even a well-placed scanner has blind spots of a different kind: surfaces it cannot measure. The beams that hit a glass pane of a laser scanner return no echo and report the maximum range, as if nothing were there. A robot that relies only on the scanner drives into glass. Ultrasonic sensors do detect glass, so robots add them to the laser (Barreto-Cubero et al. 2021). Figure 12 shows why the two differ in front of a pane.

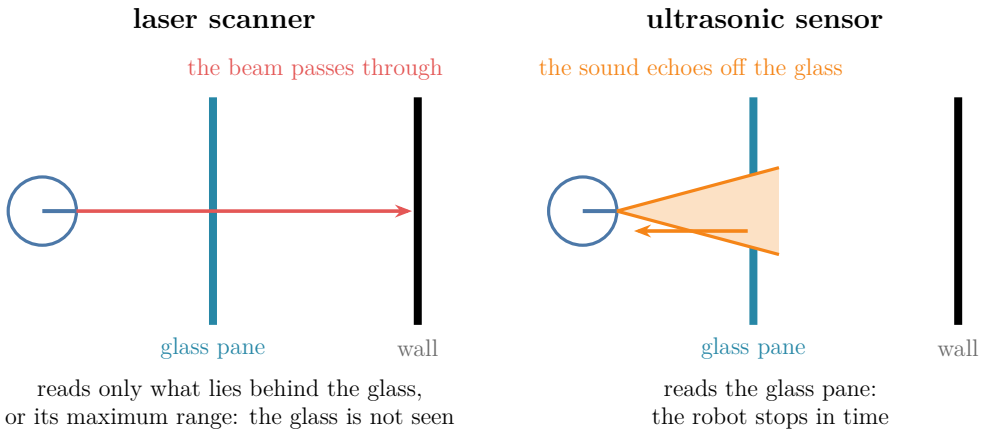


Figure 12: A laser scanner and an ultrasonic sensor in front of a glass pane with a wall behind it. Left: the laser beam passes through the glass, so the scanner reads only what lies behind it, or its maximum range: the glass is not seen. Right: the ultrasonic sensor's cone of sound echoes off the glass, so it reads the pane and the robot can stop in time

Correll gives the general reason: a sensor of one kind cannot solve a whole problem such as localization on its own; each measures something the others miss (Correll §5.1).

Adding a second sensor of a different kind, so that one sensor's blind spots or failures are covered by the other, is **sensor redundancy** (G-3219). Redundancy has a second benefit, accuracy: in the Kalman filter's corridor the commands alone missed by 0.27 m and the beacon alone by 0.51 m, and the two together by 0.17 m. Every extra sensor, though, adds data to process.

6.4 Compute budget: data the computer can process in time

More sensors and finer sensors send more data, and every reading must be processed before the next arrives. Per second:

$$682 \times 10 = 6820 \text{ (laser readings)}$$

$$1280 \times 720 \times 30 = 27.6 \text{ million (camera)}$$

$$27\,648\,000/6820 = 4054$$

The depth camera at its full 1280 × 720 resolution and 30 frames a second (RealSense D435 specification) sends 4054 times as many points as the laser scanner. Figure 13 draws the two as areas.

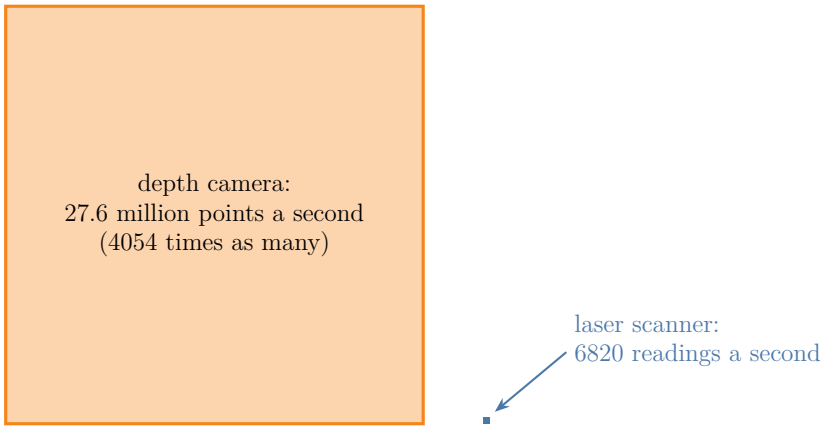


Figure 13: Readings per second drawn as areas: the depth camera's 27.6 million points a second (orange square) against the laser scanner's 6820 readings a second (the small blue square, arrowed), 4054 times fewer. Each reading must be processed before the next one arrives

The time and memory the robot's computer can spend on each sensor's data, so that every reading is processed before the next one arrives, is its **compute budget** (G-3220). A reading processed late is a reading about where the person was.

Real stacks keep within the budget by using less of the data:

- Nav2's particle filter (AMCL) uses 60 of each scan's readings (`max_beams: 60`);
- Nav2's local costmap is updated 5 times a second, not at the sensor's own rate (`update_frequency: 5.0`);

- each laser reading clears the cells along its ray, up to 3.0 m (`raytrace_max_range`) of 0.05 m cells, so one scan visits at most the cells below, and a camera frame of 921,600 points would visit far more:

$$3.0/0.05 = 60 \text{ cells per reading}$$

$$682 \times 60 = 40\,920 \text{ cells per scan}$$

The robot now has its sensors chosen to see the person in time: range 0.67 m, all round, with a second kind of sensor for glass, within what the computer can process every 0.1 s.

7. Summary

Design	Looks	In the room	Why
sense–plan–act at 1 Hz	once a second	touches the person at 3.22 s	it sees them 0.6 s late
sense–plan–act at 10 Hz	every 0.1 s	stops 0.112 m from the person	but the whole-map step cannot run that fast
vehicle 3a alone	every reading	touches the table at 9.84 s	it senses only the lamp
subsumption, two levels	every 0.1 s	safe, but stops 0.283 m from the person	pull and push cancel; no plan
layered stack	1 to 20 times a second	door at 8.0 s, 0.14 m clear	slow layers plan, fast layers react

- The opening's one loop touched the person at 3.22 s because it looked once a second and saw them 0.6 s late, with 0.113 m left and 0.350 m needed to brake; the layered stack reached the door at 8.0 s, 0.14 m clear, because its local planner saw the person in the cycle at 2.4 s and its controller turned the robot within 0.05 s.
- **Sense–plan–act** (G-3211) runs sensing, planning and acting in order, so the robot looks only once per pass; to stop for the person it must look every 0.187 s or faster, but a whole-map step runs about once a second.
- **Reactive control** (G-3212) maps readings straight to the wheels through **behaviours** (G-3213), so it reacts within one reading; a **Braitenberg vehicle** (G-3214) steers to or from a light by two wires alone, but it touches a table it cannot sense.
- The **subsumption architecture** (G-3215) runs behaviours in levels, a higher level taking over by **suppression and inhibition** (G-3216), so the robot never touches anything; but with no map or plan it stops 0.283 m from the person and stays there, because the lamp's pull and the person's push cancel.
- A **hybrid deliberative/reactive architecture** (G-3217) runs planning and reacting at the same time, so the **autonomy stack** (G-3210) can plan the route once and still react in 0.1 s: mission, behaviour, motion and control, each at the rate its inputs change.
- Nav2, Autoware and Apollo use the same layers: Nav2 re-plans globally once a second and controls at 20 Hz, Autoware plans at 10 Hz and controls every 0.03 s, Apollo plans at 10 Hz and controls every 0.01 s, so the slow layers never hold up the fast ones.
- A sensor must reach 0.67 m from the robot's centre so the robot can stop from 0.6 m/s, cover every direction the robot moves in through its **field of view** (G-3218), be backed by another kind for its blind spots through **sensor redundancy** (G-3219), and fit the **compute budget** (G-3220), because a reading processed late describes the past.

The robot can now run its parts as one layered stack with sensors chosen to see in time. One part is still missing: the layers are separate programs that must pass scans, plans and commands to each other at their own rates without waiting, which is the job of ROS 2's nodes, topics, services and actions.

8. Sources

Built from

- Brian Douglas, "The Braitenberg Vehicles", YouTube, https://www.youtube.com/watch?v=A-fxij3zM7g: the simple robot set against a complicated one that must foresee every situation, one sensor wired to one motor, the inverted wire, and the four two-sensor vehicles (fear, aggression, the one that loves the light and the one that explores)
- Chernova, S. *CS 7633 Human-Robot Interaction*, "Behavior Architectures" (slides), Georgia Institute of Technology, https://faculty.cc.gatech.edu/~chernova/CS7633/slides/BehaviorArchitectu behaviour, deliberative sense–plan–act and Shakey, reactive systems, what a purely reactive robot cannot do, when deliberative systems work, hybrid deliberative/reactive architectures (Chernova, CS 7633 slides)
- Dean, T. *CS148 Building Intelligent Robots*, "Architectures" (lecture notes), Brown University, https://cs.brown.edu/~tld/courses/cs148/02/architectures.html: Brooks's levels 0 and 1, collide, feelforce and runaway, suppression and inhibition (Dean, CS148 notes)
- Jenkins, O. C. *CSCI 1480*, "Topic 14: Subsumption Architecture" (slides), Brown University, https://cs.brown.edu/courses/csci1480/lectures/cs148_14_subsumption.pdf: subsumption as a priority chain of reactive controllers (Jenkins, CSCI 1480 slides)
- Wikipedia, "Subsumption architecture", https://en.wikipedia.org/wiki/Subsumption_architecture, and "Braitenberg vehicle", https://en.wikipedia.org/wiki/Braitenberg_vehicle (read 2026-10-12)
- Correll, N. *Introduction to Autonomous Robots*, ch. 5 "Sensors": §5.1 robotic sensors, §5.3 sensors using light, §5.4 sensors using sound, §5.7 terminology, LibreTexts edition, https://eng.libretexts.org/Bookshelves/Mechanical_Engineering/Introduction_to_Autonomous_Rob (Correll §5.1, §5.4)
- Barreto-Cubero, A. J., Gómez-Espinosa, A., Escobedo Cabello, J. A., Cuan-Urquizo, E. and Cruz-Ramírez, S. R. (2021). "Sensor data fusion for a mobile robot using neural networks". *Sensors* 22(1), 305, https://pmc.ncbi.nlm.nih.gov/articles/PMC8749872/ (Barreto-Cubero et al. 2021)
- Nav2, GitHub, https://github.com/ros-navigation/navigation2 (main, read 2026-10-12): `nav2_bringup/params/nav2_params.yaml` (nav2_params.yaml) and `nav2_bt_navigator/behavior_trees/navigate_to_pose_w_replanning_and_recovery.xml` (Nav2 default tree)
- Autoware, the launch and parameter repository on GitHub, https://github.com/autowarefoundation/au (main, read 2026-10-12): `autoware_universe_launch/autoware_planning_config/config/scenar` (`planning_hz`) and `autoware_universe_launch/autoware_control_config/config/trajectory_` (`ctrl_period`); Autoware documentation, planning component design, https://github.com/autowarefo documentation, `docs/design/autoware-architecture-v1/components/planning/index.md` (Autoware planning design)
- Apollo, GitHub, https://github.com/ApolloAuto/apollo (master, read 2026-10-12): `modules/planning/planning_base/gflags/planning_gflags.cc` (`planning_loop_rate`), `modules/control/control_component/dag/control.dag` and `conf/control.conf`
- Hokuyo, URG-04LX-UG01 product page and specification, https://www.hokuyo-

aut.jp/search/single.php?serial=166 (Hokuyo specification)
- RealSense, Depth Camera D435 product page and specification, https://www.realsenseai.com/products camera-d435/ (RealSense D435 specification)
- ROS 2, `sensor_msgs/msg/LaserScan.msg`, https://github.com/ros2/common_interfaces (sensor_msgs/LaserScan)

Other references

- Brooks, R. A. (1986). "A robust layered control system for a mobile robot". *IEEE Journal of Robotics and Automation* 2(1), 14–23 (Brooks 1986): the original subsumption paper; its levels are read here through Dean's notes
- Braitenberg, V. (1984). *Vehicles: Experiments in Synthetic Psychology*. MIT Press: the book of the vehicles, read here through Wikipedia's summary

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Sense–plan–act (deliberative control) (G-3211)	A way to run a robot in one fixed loop: read the sensors, update a model of the world and plan, then act on the plan, and repeat; the robot looks again only when the whole loop is done, so the slowest step, such as planning over the whole map, sets how often it can react.
Reactive control (G-3212)	Control that maps the present sensor readings straight to motor commands, with no model of the world and no plan; the robot reacts within one reading, but it cannot plan a route, build a map or choose between behaviours.
Behaviour (robotics) (G-3213)	One mapping from sensor readings to motor actions, made for one task, such as "turn towards the light" or "halt if something is close ahead"; reactive and behaviour-based robots are built by combining several.
Braitenberg vehicle (G-3214)	A vehicle whose one or two light sensors are wired straight to its wheels, each wire speeding its wheel up or slowing it down with more light, on the same side or crossed; the wiring alone makes it flee a light, charge at it, stop near it or pass it by, with no plan.

Term	Meaning
Subsumption architecture (G-3215)	A design, due to Brooks (1986), in which behaviours run in parallel in levels, the lowest such as “avoid” built and tested first; a higher level adds a competence such as “go to the light” and takes over the lower levels’ commands when it has one, while safety reflexes stay in force.
Suppression and inhibition (G-3216)	The two ways a higher subsumption level takes over a lower one: suppression replaces a lower module’s input or command with the higher level’s own signal, and inhibition blocks a lower module’s output from going anywhere; they decide which behaviour drives the wheels.
Hybrid deliberative/reactive architecture (G-3217)	A robot design that runs a slow planning part, which reasons over a model of the world, and a fast reactive part, which acts on the latest readings, at the same time; the robot keeps a plan and still reacts within one fast cycle.
Autonomy stack (G-3210)	The whole of a robot’s software arranged in layers, sensing at the bottom and mission, behaviour, motion and control above it, each layer running at its own rate and working from the latest output of the layer above; the slow layers plan far ahead while the fast ones react, so a new reading reaches the wheels in one fast cycle.
Field of view (G-3218)	The directions a sensor can measure, as an angle around it, such as 240° for a laser scanner or 87° for a depth camera; with the range it decides what the sensor covers, and outside it the sensor measures nothing.
Sensor redundancy (G-3219)	Adding a sensor of a different kind that measures the same thing, such as ultrasonic sensors beside a laser scanner, so that one sensor’s blind spots or failures (glass for the laser) are covered by the other; combined, they are also more accurate.

Term	Meaning
Compute budget (G-3220)	The computing time and memory a robot can spend on each sensor's data so that every reading is processed before the next one arrives; a sensor that sends more than the budget allows gives the planner readings that are already out of date, so stacks use less of the data.
A* search (G-2577)	Dijkstra's algorithm with the open list sorted by $f = g + h$, the cost so far plus a heuristic guess of the cost left; with a consistent heuristic it returns the shortest route while expanding far fewer states (9 cells against 54 on the room grid).
Particle filter (G-2492)	A Bayes filter that stores the belief as a set of weighted samples (particles) and runs each round as move, weigh, resample; it can hold beliefs of any shape, such as a belief with three modes, one at each of three doors.
Local planner (G-2911)	The planner that turns the next few metres or seconds of the global plan into a timed trajectory within the robot's limits and around obstacles its sensors see, solving again many times a second, such as TEB at 10 Hz over 1.5 m; it handles what the global plan cannot.
Angular velocity ω (G-2292)	How fast a robot's heading changes, in rad/s, positive to the left; for a differential drive, the difference of the wheel speeds divided by the wheel separation.
Route planning (mission planning) (G-2870)	Finding the best way from where a vehicle is to its destination on a map of the road network, by travel time or by distance, such as the 492.8 s route across the Chicago map; the top level of planning, which picks the roads and lanes and leaves the motion to the levels below.
Behavioural layer (behavioural decision making) (G-2912)	The level of planning between route planning and motion planning that decides which task the vehicle carries out now, such as following the lane, stopping at a line or giving way, from the route and what it perceives; the motion planner then plans that task.

Term	Meaning
Global planner (G-2910)	The planner that finds a route over the whole map, from the robot to its goal, and runs rarely, such as A* on a room's grid or a road network; its route, the global plan, guides the local planner but has no timing and includes only the obstacles on the map.
Controller (G-2947)	The part of a feedback loop that turns the control error into a command, such as a computer running "push 1 N plus 20 N per m/s of error"; it decides how hard to correct.
Maximum range $z_{\max}$ (G-2373)	The longest distance a range sensor can report, such as 5 m; when no pulse returns, the sensor reports this value, so a reading of $z_{\max}$ usually means a failure, not a wall.